Kernel debugging



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Debugging/tracing using logs 1/4

- Good old printk()!
  - Works in all contexts
  - Can specify a log level ranging from `0` (emergency) to `7` (debug)
  - Be careful of the delays introduced when logs are printed out on a serial console at 115200 bauds
    - A `*_ratelimited()` version exists to limit the amount of print if called too often
  - Not recommended for upstream contributions

Example:

```
printk("in probe\n");
```

Here's what you get in the kernel log:

```
[    1.878382] in probe
```

All other logging facilities are based on it.



Debugging/tracing using logs 2/4

- The `pr_*()` family of functions
  - They include the log level in the name: pr_emerg(), pr_alert(), pr_crit(), pr_err(), pr_warn(), pr_notice(), pr_info(), pr_cont() and the special pr_debug() (see next pages)
  - They allow setting a manual prefix (eg. eases grepping):

```
#define pr_fmt(fmt) "foo: " fmt
```

- Also defined in include/linux/printk.h

Example:

```
pr_info("in probe\n");
```

Here's what you get in the kernel log:

```
[    1.878382] in probe
```

or similarly with a manual format:

```
[    1.878382] foo: in probe
```



Debugging/tracing using logs 3/4

- The `dev_*()` family of functions
  - They include a formatted standard prefix with the device name: dev_emerg(), dev_alert(), dev_crit(), dev_err(), dev_warn(), dev_notice(), dev_info() and the special dev_dbg() (see next pages)
  - They additionally take a pointer to struct device as first argument
  - Defined in include/linux/dev_printk.h
  - To be used in device drivers

Example:

```
dev_info(&pdev->dev, "in probe\n");
```

Here's what you get in the kernel log:

```
[    1.878382] serial 48024000.serial: in probe
[    1.884873] serial 481a8000.serial: in probe
```



Debugging/tracing using logs 4/4

- The kernel defines many more format specifiers than the standard `printf()` existing ones.
  - `%p`: Display the hashed value of pointer by default
  - `%px`: Always display the address of a pointer (use carefully on non-sensitive)
  - `%pK`: Display hashed pointer value, zeros or the pointer address depending on `kptr_restrict` sysctl value
  - `%pOF`: Device-tree node format specifier
  - `%pr`: Resource structure format specifier
  - `%pa`: Physical address display (work on all architectures 32/64 bits)
  - `%pe`: Error pointer (displays the string corresponding to the error number)
- See core-api/printk-formats for an exhaustive list of format specifiers
- Also features a helper to dump entire buffers with a `hexdump` like display: print_hex_dump()



pr_debug() and dev_dbg()

- When the driver is compiled with `DEBUG` defined, all these messages are compiled and printed at the debug level. `DEBUG` can be defined at the beginning of the driver, or using `ccflags-$(CONFIG_DRIVER) += -DDEBUG` in the `Makefile`
- When the kernel is compiled with CONFIG_DYNAMIC_DEBUG, then these messages can dynamically be enabled on a per-file, per-module or per-message basis, by writing commands to `/proc/dynamic_debug/control`. Note that messages are not enabled by default.
  - Details in admin-guide/dynamic-debug-howto
  - Very powerful feature to only get the debug messages you're interested in.
- When neither `DEBUG` nor CONFIG_DYNAMIC_DEBUG are used, these messages are not compiled in.



Configuring the priority

- Each message is associated to a priority, as specified in include/linux/kern_levels.h.
- All the messages, regardless of their priority, are stored in the kernel log ring buffer
  - Typically accessed using the `dmesg` command
- Messages with a priority lower than the `loglevel` also appear on the console
- The `loglevel` can be changed:
  - in the kernel configuration using CONFIG_CONSOLE_LOGLEVEL_DEFAULT)
  - on the cmdline with `loglevel=` (see admin-guide/kernel-parameters)
  - at runtime through `/proc/sys/kernel/printk` (admin-guide/sysctl/kernel)
- Examples:
  - `loglevel=0`: no message on the console (see also: `quiet`)
  - `loglevel=8`: all messages on the console (see also: `ignore_loglevel`)



DebugFS

A virtual filesystem to export debugging information to user space.

- Kernel configuration: CONFIG_DEBUG_FS
  - `Kernel hacking -> Debug Filesystem`
- The debugging interface disappears when Debugfs is configured out.
- You can mount it as follows:
  - `sudo mount -t debugfs none /sys/kernel/debug`
- First described on https://lwn.net/Articles/115405/
- API documented in the Linux Kernel Filesystem API: filesystems/debugfs The debugfs filesystem



DebugFS API

- Create a sub-directory for your driver:

```
struct dentry *debugfs_create_dir(const char *name,
                                  struct dentry *parent);
```

- Expose an integer as a file in DebugFS. Example:

```
struct dentry *debugfs_create_u8(const char *name, mode_t mode,
                                 struct dentry *parent, u8 *value);
```

- u8, u16, u32, u64 for decimal representation
- x8, x16, x32, x64 for hexadecimal representation
- Expose a binary blob as a file in DebugFS:

```
struct dentry *debugfs_create_blob(const char *name, mode_t mode,
                                   struct dentry *parent,
                                   struct debugfs_blob_wrapper *blob);
```

- Also possible to support writable DebugFS files or customize the output using the more generic debugfs_create_file() function.



Using Magic SysRq

Functionality provided by serial drivers

- Allows to run multiple debug / rescue commands even when the kernel seems to be in deep trouble
  - On PC: press `[Alt]` + `[Prnt Scrn]` + `<character>` simultaneously (`[SysRq]` = `[Alt]` + `[Prnt Scrn]`)
  - On embedded: in the console, send a break character (Picocom: press `[Ctrl]` + `a` followed by `[Ctrl]` + `\`), then press `<character>`
- Example commands:
  - `h`: show available commands
  - `s`: sync all mounted filesystems
  - `b`: reboot the system
  - `n`: makes RT processes nice-able.
  - `w`: shows the kernel stack of all sleeping processes
  - `t`: shows the kernel stack of all running processes
  - You can even register your own!
- Detailed in admin-guide/sysrq



kgdb - A kernel debugger

- CONFIG_KGDB in *Kernel hacking*.
- The execution of the kernel is fully controlled by `gdb` from another machine, connected through a serial line.
- Can do almost everything, including inserting breakpoints in interrupt handlers.
- Feature supported for the most popular CPU architectures
- CONFIG_GDB_SCRIPTS allows to build GDB python scripts that are provided by the kernel.
  - See process/debugging/kgdb for more information



Using kgdb (1/2)

- Details available in the kernel documentation: process/debugging/kgdb
- You must include a kgdb I/O driver. One of them is `kgdb` over serial console (`kgdboc`: `kgdb` over console, enabled by CONFIG_KGDB_SERIAL_CONSOLE)
- Configure `kgdboc` at boot time by passing to the kernel:
  - `kgdboc=<tty-device>,<bauds>`.
  - For example: `kgdboc=ttyS0,115200`
- Or at runtime using sysfs:
  - `echo ttyS0 > /sys/module/kgdboc/parameters/kgdboc`
  - If the console does not have polling support, this command will yield an error.



Using kgdb (2/2)

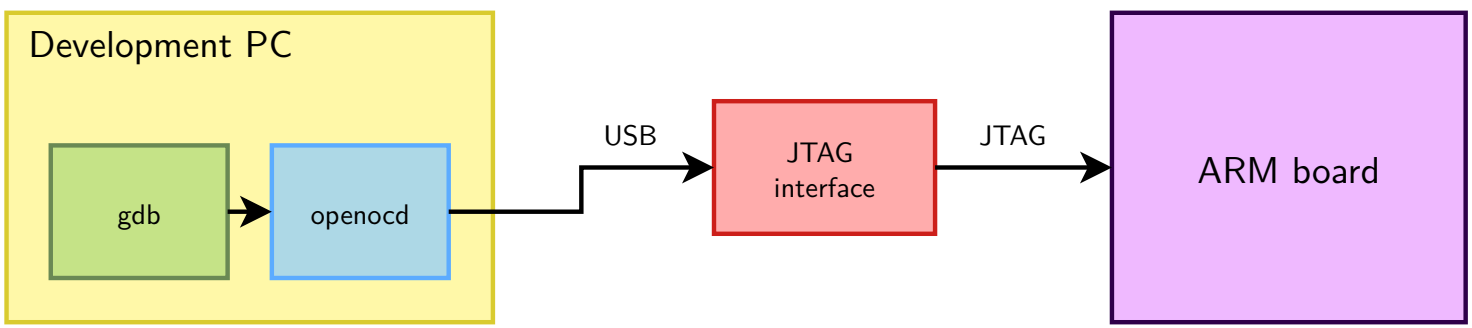
- Then also pass `kgdbwait` to the kernel: it makes `kgdb` wait for a debugger connection.
- Boot your kernel, and when the console is initialized, interrupt the kernel with a break character and then `g` in the serial console (see our *Magic SysRq* explanations).
- On your workstation, start `gdb` as follows:
  - `arm-linux-gdb ./vmlinux`
  - `(gdb) set serial baud 115200`
  - `(gdb) target remote /dev/ttyS0`
- Once connected, you can debug a kernel the way you would debug an application program.
- On GDB side, the first threads represent the CPU context (ShadowCPU<x>), then all the other threads represents a task.



Debugging with a JTAG interface

Two types of JTAG dongles

- The ones offering a `gdb` compatible interface, over a serial port or an Ethernet connection. `gdb` can directly connect to them.
- The ones not offering a gdb compatible interface are generally supported by OpenOCD (Open On Chip Debugger): https://openocd.sourceforge.net/
  - OpenOCD is the bridge between the gdb debugging language and the JTAG interface of the target CPU.
  - See the very complete documentation: https://openocd.org/pages/documentation.html
  - For each board, you'll need an OpenOCD configuration file (ask your supplier)





Early traces

- If something breaks before the `tty` layer, serial driver and serial console are properly registered, you might just have nothing else after "`Starting kernel...`"
- On ARM, if your platform implements it, you can activate (CONFIG_DEBUG_LL and CONFIG_EARLY_PRINTK), and add `earlyprintk` to the kernel command line
  - Assembly routines to just push a character and wait for it to be sent
  - Extremely basic, but is part of the uncompressed section, so available even if the kernel does not uncompress correctly!
- On other platforms, hoping that your serial driver implements OF_EARLYCON_DECLARE(), you can enable CONFIG_SERIAL_EARLYCON
  - The kernel will try to hook an appropriate `earlycon` UART driver using the `stdout-path` of the device-tree.



More kernel debugging tips 1/2

- Make sure CONFIG_KALLSYMS_ALL is enabled
  - To get oops messages with symbol names instead of raw addresses
  - Turned on by default
- Make sure CONFIG_DEBUG_INFO is also enabled
  - This way, the kernel is compiled with `$(CROSSCOMPILE)gcc -g`, which keeps the source code inside the binaries.
- If your device is not probed, try enabling CONFIG_DEBUG_DRIVER
  - Extremely verbose!
  - Will enable all the debug logs in the device-driver core section



More kernel debugging tips 2/2

Device Tree output can be better understood with the in-tree `dtx_diff` script

- Show the origin of each property/node: `scripts/dtc/dtx_diff -T <dts>`

```
main_i2c3: i2c@20030000 { /* arch/arm64/boot/dts/ti/k3-am62-main.dtsi:450:26-460:4,
                            arch/arm64/boot/dts/ti/k3-am625-beagleplay.dts:820:12-825:3,
                            arch/arm64/boot/dts/ti/k3-am625-beagleplay-custom.dts:20:12-25:3 */
    ...
    reg = <0x00 0x20030000 0x00 0x100>; /* arch/arm64/boot/dts/ti/k3-am62-main.dtsi:452:3-452:38 */
    status = "okay"; /* arch/arm64/boot/dts/ti/k3-am625-beagleplay.dts:824:2-824:18 */

    joystick@52 { /* arch/arm64/boot/dts/ti/k3-am625-beagleplay-custom.dts:21:14-24:4 */
        compatible = "nintendo,nunchuk"; /* arch/arm64/boot/dts/ti/k3-am625-beagleplay-custom.dts:23:3-23:35 */
        reg = <0x52>; /* arch/arm64/boot/dts/ti/k3-am625-beagleplay-custom.dts:22:3-22:16 */
    };
};
```

- Can also be used to diff DTS



Getting help and reporting bugs

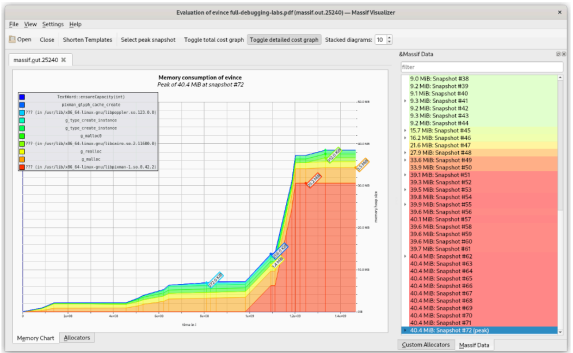
- If you are using a custom kernel from a hardware vendor, contact that company. The community will have less interest supporting a custom kernel.
- Otherwise, or if this doesn't work, try to reproduce the issue on the latest version of the kernel.
- Make sure you investigate the issue as much as you can: see admin-guide/bug-bisect
- Check for previous bugs reports. Use web search engines, accessing public mailing list archives.
- If you're the first to face the issue, it's very useful for others to report it, even if you cannot investigate it further.
- If the subsystem you report a bug on has a mailing list, use it. Otherwise, contact the official maintainer (see the MAINTAINERS file). Always give as many useful details as possible.



Debugging resources

Checkout Bootlin's debugging training!

- Linux debugging, profiling, tracing and performance analysis training
- https://bootlin.com/doc/training/debugging/
- Slides and labs are available for free



Practical lab - Kernel debugging



- Use the dynamic debug feature.
- Add debugfs entries
- Load a broken driver and see it crash
- Analyze the error information dumped by the kernel.
- Disassemble the code and locate the exact C instruction which caused the failure.